第4课：序列模型

- 李伟昌, weichangli@zju.edu.cn
- 浙江大学**海洋人工智能**课程讲义

- 版本 1, 2026年5月20日

时长： 45分钟
目标： 理解序列数据建模、RNN和注意力基础

4.1 序列为何不同

序列数据示例

- **自然语言：** 句子、文档、对话
- **时间序列：** 股票价格、天气、传感器数据
- **音频：** 语音、音乐
- **生物：** DNA序列、蛋白质结构

挑战

1. **可变长度：** 不同序列有不同长度
2. **时间依赖：** 顺序很重要（"狗咬人" ≠ "人咬狗"）
3. **长距离依赖：** 来自遥远过去的上下文可能重要
4. **没有固定输入大小：** 不像固定维度的图像

为什么不使用标准神经网络?

- 仅固定大小输入
- 跨时间没有参数共享
- 无法建模时间结构

4.2 循环神经网络

核心思想

维护隐藏状态$\mathbf{h}_t$，总结到时间t的信息。

RNN方程

在每个时间步t：

$$\mathbf{h}_t = f(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$$

$$\mathbf{y}_t = g(\mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y)$$

其中：

- $\mathbf{x}_t$：时间t的输入
- $\mathbf{h}_t$：隐藏状态（记忆）
- $\mathbf{y}_t$：时间t的输出
- f：激活（通常tanh或ReLU）
- g：输出激活（分类用softmax）

计算图（展开）

```
时间:    t=1         t=2         t=3         t=4
          │           │           │           │
          ▼           ▼           ▼           ▼
         ┌─┐         ┌─┐         ┌─┐         ┌─┐
x_1 ───►│R│──y_1    │ │         │ │         │ │
         │N│         │ │         │ │         │ │
         │N├──h_1───►│R│──y_2    │ │         │ │
         └─┘         │N│         │ │         │ │
                     │N├──h_2───►│R│──y_3   │ │
                     └─┘         │N│        │ │
                                 │N├──h_3───►│R│──y_4
                                 └─┘         └─┘


RNN单元在每个时间步:
                      h_{t-1}
                         │
                         ▼
    x_t ───┬──►[W_xh]─────┴──►[+]──►[tanh]──► h_t
           │                   ▲
           └──►[W_hh]──────────┘
                         (每个时间步相同的W_hh! )
```

每个时间步相同的参数$\mathbf{W}_{hh}, \mathbf{W}_{xh}, \mathbf{W}_{hy}$（参数共享）。

Karpathy的"Vanilla"RNN单元（micrograd风格视图）：

```
    ┌──────────────────────────────────────────┐
    │  tanh( Wxh @ xt  +  Whh @ ht-1  +  b )   │
    │        ▲               ▲                 │
    │        │               │                 │
    └────────┼───────────────┼─────────────────┘
             │               │
           Input           来自
           at t            上一步
                           的记忆
```

PyTorch实现

```
class SimpleRNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()
        self.hidden_size = hidden_size
        self.i2h = nn.Linear(input_size + hidden_size, hidden_size)
        self.i2o = nn.Linear(input_size + hidden_size, output_size)
        self.tanh = nn.Tanh()

    def forward(self, input_seq):
        hidden = torch.zeros(1, self.hidden_size)
        outputs = []

        for x_t in input_seq:
            combined = torch.cat((x_t, hidden), dim=1)
            hidden = self.tanh(self.i2h(combined))
            output = self.i2o(combined)
            outputs.append(output)

        return torch.stack(outputs), hidden
```

RNN变体

一对一： 标准NN（图像 → 标签） **一对多：** 图像描述（图像 → 序列） **多对一：** 情感分析（序列 → 情感） **多对多：** 翻译（序列 → 序列） **多对多（同步）：** 帧级视频标注

4.3 梯度消失问题

问题

通过时间反向传播：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_t \frac{\partial \mathcal{L}_t}{\partial \mathbf{W}_{hh}}$$

每一步涉及：

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \mathbf{W}_{hh}^T \cdot \mathrm{diag}(f'(\mathbf{z}_t))$$

当反向传播T步时：

$$\frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_1} = \prod_{t=1}^{T} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}}$$

问题： T项的矩阵积 →

- 如果特征值 < 1：梯度**指数级消失**
- 如果特征值 > 1：梯度**指数级爆炸**

后果： RNN难以处理长距离依赖。

解决方案

1. **梯度裁剪：** 限制梯度大小（解决爆炸）
2. **更好的架构：** LSTM、GRU（解决消失）
3. **注意力机制：** 直接连接（下一节）

```
# PyTorch中的梯度裁剪
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
```

4.4 LSTM和GRU

长短期记忆（LSTM）- Hochreiter & Schmidhuber (1997)

核心思想： 显式门控机制控制信息流。

LSTM单元结构 - 视觉分解

```
         ┌────────────────────────────────────┐
         │              LSTM单元在时间t        │
         └────────────────────────────────────┘

    x_t ──┬────────────────────────────────────┐
          │                                    │
          ▼                                    ▼
   ┌────────┐   ┌────────┐   ┌────────┐   ┌────────┐
   │  遗忘   │  │  输入   │  │  输出   │  │ 候选    │
   │   门    │  │   门    │  │   门    │  │  值     │
   │  σ(·)  │   │  σ(·)  │   │  σ(·)  │   │ tanh(·) │
   └────────┘   └────────┘   └────────┘   └────────┘
       │            │            │            │
       ▼            ▼            │            ▼
    f_t (0-1)    i_t (0-1)       │         Ĉ_t (-1,1)
       │            │            │            │
       │       ┌────┘            │            │
       │       │                 │            │
C_{t-1}───►[×]◄──────[×]◄────────┴────────────┘
(前一)        │        ▲ (新信息)    o_t
记忆        遗忘      添加新
              │        │
              └───┬────┘
                  ▼
            ┌────────┐
            │  tanh   │◄─────────────────┐
            └────────┘                   │
                │                        │
                ▼                        │
            ┌────────┐                   │
            │   ×     │◄─────────────────┘
            └────────┘
                │
                ▼
              h_t (输出)
                │
                ▼
            C_t (细胞状态)
            (到下一步)
```

遗忘门： 从细胞状态中丢弃什么

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

输入门： 存储什么新信息

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_C \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_C)$$

更新细胞状态（"传送带"）：

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$$

输出门： 输出什么

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t)$$

关键特性

- **加法更新**到细胞状态（非乘法）
- 门控制信息流
- **跳跃连接**通过时间（细胞状态）
- 可以长期保持信息

```
# PyTorch LSTM
lstm = nn.LSTM(input_size=100, hidden_size=128, num_layers=2,
               batch_first=True, dropout=0.3, bidirectional=True)

# forward返回：output, (hidden, cell)
output, (hn, cn) = lstm(x)  # x形状：(batch, seq_len, input_size)
```

门控循环单元（GRU）- Cho等人 (2014)

LSTM的简化版本，门更少：

更新门： z_t（保留多少过去）

重置门： r_t（遗忘多少过去）

候选激活：

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W} \cdot [\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t])$$

最终隐藏状态：

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$$

LSTM vs GRU

特性	LSTM	GRU
门	3（遗忘、输入、输出）	2（更新、重置）
细胞状态	分离	与隐藏状态合并
参数	更多	更少（~25%）
性能	相似	相似
何时使用	非常长的序列	资源受限

4.5 词嵌入

为什么不用One-Hot?

词汇量V = 50,000+

- One-hot：50,000维
- 没有语义关系

目标： 相似词接近的密集向量。

Word2Vec - Mikolov等人 (2013)

Skip-gram： 从词预测上下文

$$P(w_{context} | w_{target})$$

CBOW： 从上下文预测词

$$P(w_{target} | w_{context})$$

关键见解： 出现在相似上下文中的词具有相似含义。

特性：

- King - Man + Woman ≈ Queen
- Paris - France + Italy ≈ Rome

GloVe - Pennington等人 (2014)

使用全局共现统计：

$$\mathbf{w}_i^T \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j = \log(X_{ij})$$

捕获局部和全局信息。

使用预训练嵌入

```
# PyTorch嵌入层
embedding = nn.Embedding(num_embeddings=50000, embedding_dim=300)

# 加载预训练GloVe
glove_weights = torch.load('glove.6B.300d.pt')
embedding.weight.data.copy_(glove_weights)
embedding.weight.requires_grad = False  # 冻结或设为True用于微调

# 将词索引转换为嵌入
word_indices = torch.LongTensor([1, 45, 892, 3])  # 词ID批次
embedded = embedding(word_indices)  # (seq_len, 300)
```

4.6 序列到序列模型

架构

编码器： RNN处理输入序列 → 最终隐藏状态 **解码器：** RNN从编码生成输出序列

```
输入:  "Hello" → "world" → "!"
          ↓          ↓        ↓
编码器:  [RNN] → [RNN] → [RNN] → 上下文
                                    ↓
解码器:  [SOS] → "Bonjour" → "le" → "monde" → [EOS]
```

训练 vs 推理

训练（Teacher Forcing）：

- 解码器在每个时间步接收真实值作为输入
- 更快收敛

推理：

- 解码器接收自己的预测作为下一个输入
- 可以使用束搜索获得更好结果

局限性

- **瓶颈：** 所有信息压缩到单个向量
- **长序列：** 编码器难以捕获所有内容
- **解决方案：** 注意力机制（下一课）

4.7 注意力介绍

动机

不是单个上下文向量，让解码器"查看"所有编码器隐藏状态。

注意力机制

在每个解码器时间步t：

1. 计算对齐分数：$e_{t,s} = \mathrm{score}(s_t, h_s)$
2. 计算注意力权重：$\alpha_{t,s} = \frac{\exp(e_{t,s})}{\sum_{s'} \exp(e_{t,s'})}$
3. 计算上下文向量：$c_t = \sum_s \alpha_{t,s} h_s$

分数函数

- **点积：** $e_{t,s} = s_t^T h_s$
- **加法：** $e_{t,s} = v^T \tanh(W_s s_t + W_h h_s)$
- **缩放点积：** $e_{t,s} = \frac{s_t^T h_s}{\sqrt{d_k}}$（Transformers中使用）

预览： 这个机制成为Transformer架构的基础。

4.8 实验：处理序列

练习1：字符级RNN

训练一个字符级LSTM生成文本：

1. 处理文本语料库
2. 构建字符词汇表
3. 训练LSTM预测下一个字符
4. 通过采样生成新文本

```
class CharRNN(nn.Module):
    def __init__(self, vocab_size, hidden_size):
        super().__init__()
        self.hidden_size = hidden_size
        self.embedding = nn.Embedding(vocab_size, hidden_size)
        self.lstm = nn.LSTM(hidden_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, x, hidden=None):
        x = self.embedding(x)
        out, hidden = self.lstm(x, hidden)
        out = self.fc(out)
        return out, hidden
```

练习2：情感分析

使用预训练嵌入+LSTM进行情感分类：

1. 加载GloVe嵌入
2. 处理IMDB或类似数据集
3. 训练多对一LSTM分类器
4. 比较有/无预训练嵌入

练习3：Seq2Seq翻译

实现一个简单的英法翻译器：

- 带LSTM的编码器-解码器
- 添加注意力机制
- 比较有/无注意力质量

关键要点

1. RNN维护隐藏状态以处理可变长度的序列
2. Vanilla RNN遭受梯度消失 → LSTM/GRU用门控修复
3. 词嵌入提供密集、语义表示
4. Seq2Seq模型使用编码器-解码器架构进行翻译/生成
5. 注意力允许解码器关注相关编码器状态
6. 这些概念直接导致Transformer架构

延伸阅读

核心材料

1. Understanding LSTM Networks - Christopher Olah（本讲改编的著名LSTM图）
2. Neural Machine Translation by Jointly Learning to Align and Translate - 注意力论文
3. Word2Vec Explained

Andrej Karpathy的精华文章

4. **The Unreasonable Effectiveness of RNNs** - 一篇经典博客文章，展示RNN在文本生成方面的惊人能力。这些笔记中的展开计算图可视化遵循Karpathy的直观解释。

5. **Neural Networks: Zero to Hero** - 第4讲（"becoming a backprop ninja"）和第5讲（makemore）提供字符级语言模型的动手实现。这些笔记中的RNN单元图受Karpathy的逐步micrograd风格解释启发。